4to Laboratorio Tecnologías de microprocesamiento

Lujan Negrin, Sury Gonzalez, Gonzalo Gorosito
Universidad Tecnológica del Uruguay, ITRSO
Fray Bentos, Uruguay
{maria.negrin.a, sury.gonzalez, gonzalo.gorosito}@estudiantes.utec.edu.uy

I. Introducción

Los sistemas embebidos modernos rara vez operan de forma aislada: deben identificar usuarios, intercambiar información con otros dispositivos, interpretar sensores de distinta naturaleza y conservar datos entre sesiones de uso. En el ATmega328P, estas capacidades se resuelven mediante la combinación de sus periféricos internos — SPI, TWI (I2C), USART, ADC, temporizadores y memoria EEPROM— con módulos externos que implementan sus propios protocolos de comunicación.

En este cuarto laboratorio se abordan cinco problemas que integran dichos recursos en aplicaciones prácticas programadas en lenguaje C. El Problema A emula un cajero automático que identifica tarjetas MIFARE mediante un lector RFID RC522, autentica al usuario por PIN y almacena de forma persistente las cuentas en EEPROM. El Problema B simula una línea de producción con banda transportadora, en la que dos microcontroladores se comunican por SPI bajo una arquitectura maestro–esclavo. El Problema C implementa un sistema de alarma antirrobo supervisado de forma inalámbrica a través de un módulo Bluetooth HC-05. El Problema D adapta el juego "Simón Dice" a una matriz LED RGB WS2812B controlada con un joystick, y el Problema E construye un nivel de burbuja digital a partir de la inclinación medida por un sensor inercial MPU6050.

En todos los casos, el comportamiento del sistema se modela mediante máquinas de estados finitos, los programas se validan en simulación antes de su implementación física y el desarrollo se versiona en un repositorio de GitHub. El presente informe describe los fundamentos teóricos involucrados, los materiales empleados, la metodología de diseño de cada solución y los resultados obtenidos.

II. Resumen

Este informe presenta el desarrollo de cinco sistemas embebidos basados en el microcontrolador ATmega328P, programados en lenguaje C, que integran comunicación con dispositivos externos, lectura de sensores, control de actuadores y almacenamiento persistente de datos. Se implementaron un cajero automático con identificación de tarjetas MIFARE mediante un lector RFID RC522, autenticación por PIN y registro de cuentas en EEPROM; una banda transportadora controlada por dos microcontroladores comunicados por SPI en una arquitectura maestro–esclavo con confirmación de comandos; un sistema de alarma antirrobo supervisado de forma inalámbrica a través de un módulo Bluetooth HC-05; un juego de memoria de secuencias sobre una matriz de LEDs RGB WS2812B controlada con un joystick; y un nivel de burbuja digital que estima la inclinación a partir de un sensor inercial MPU6050 conectado por I2C. El comportamiento de cada sistema se modeló mediante máquinas de estados finitos implementadas en un lazo principal no bloqueante, y se aplicaron técnicas de acondicionamiento de señales, como el promedio de muestras, la histéresis y la zona muerta. Los programas se desarrollaron de forma incremental, con control de versiones en GitHub, y se validaron en simulación con Proteus y PICSimLab antes de su implementación en hardware.

***Palabras clave*—ATmega328P, máquina de estados finitos, RFID, SPI, I2C, Bluetooth, EEPROM, WS2812B, MPU6050, lenguaje C.**

III. Objetivos

III-A. Objetivo general

Desarrollar sistemas embebidos basados en el microcontrolador ATmega328P, programados en lenguaje C, que integren comunicación cableada e inalámbrica con dispositivos externos, lectura de sensores, control de actuadores y almacenamiento persistente de datos.

III-B. Objetivos específicos

- Implementar la comunicación del ATmega328P con módulos externos y con otros microcontroladores mediante los protocolos SPI, I2C, UART sobre Bluetooth y el protocolo de un hilo de los LEDs WS2812B.
- Emplear la memoria EEPROM interna como medio de almacenamiento persistente de datos de usuario, puntuaciones y parámetros de calibración.

- Aplicar técnicas de filtrado digital y zona muerta en el acondicionamiento de las lecturas de sensores analógicos e inerciales.

IV. Marco Teórico

IV-A. *Interfaz SPI*

La interfaz serie periférica (SPI) es un bus síncrono, full-duplex y de tipo maestro–esclavo que utiliza cuatro líneas: MOSI (datos del maestro al esclavo), MISO (datos del esclavo al maestro), SCK (reloj generado por el maestro) y $\overline{SS}$ (selección del esclavo, activa en bajo). Cada transferencia consiste en el intercambio simultáneo de un byte entre los registros de desplazamiento de ambos dispositivos. En el ATmega328P, el módulo se configura mediante el registro SPCR (habilitación, rol maestro/esclavo, polaridad y fase del reloj, prescaler), el estado de la transferencia se consulta en SPSR (bandera SPIF) y el dato se escribe y lee en SPDR [1]. En este laboratorio, SPI se utiliza tanto para comunicar el microcontrolador con el lector RC522 como para el enlace entre los dos ATmega328P del Problema B.

IV-B. *Tecnología RFID y tarjetas MIFARE*

La identificación por radiofrecuencia (RFID) permite leer información almacenada en una etiqueta sin contacto físico, mediante acoplamiento inductivo entre la antena del lector y la de la etiqueta. Las tarjetas MIFARE Classic operan a 13,56 MHz según la norma ISO/IEC 14443-A y poseen un identificador único (UID) de 4 bytes grabado de fábrica, además de una memoria de 1 KB organizada en sectores protegidos por claves [2]. El módulo RC522, mostrado en la Fig. 1 junto con las tarjetas utilizadas, se basa en el circuito integrado MFRC522, que implementa la capa física y el protocolo de esta norma y se comunica con el microcontrolador mediante SPI. El acceso se realiza escribiendo y leyendo sus registros internos para enviar comandos a la tarjeta y recibir sus respuestas, como la solicitud de presencia (REQA) y el procedimiento anticolisión del que se obtiene el UID [3].

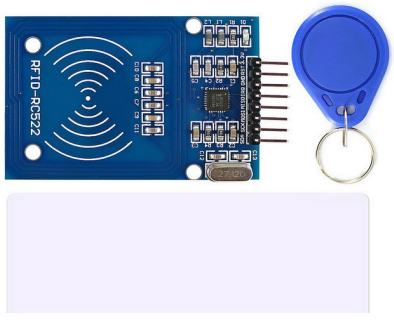


Figura 1: Lector RFID RC522 y tarjetas MIFARE Classic 1K.

IV-C. *Memoria EEPROM*

El ATmega328P dispone de 1 KB de memoria EEPROM no volátil, con una vida útil del orden de 100 000 ciclos de escritura/borrado por celda [1]. El acceso se realiza a través de los registros EEAR (dirección), EEDR (dato) y EECR (control), y cada escritura demora aproximadamente 3,3 ms. La biblioteca `avr/eeprom.h` ofrece funciones que encapsulan esta secuencia; en particular, las funciones `eeprom_update_*` solo escriben cuando el valor nuevo difiere del almacenado, lo que reduce el desgaste de la memoria. En este laboratorio la EEPROM almacena las cuentas del cajero, el récord del juego de memoria y el offset de calibración del sensor inercial.

IV-D. *Módulo Bluetooth HC-05*

El módulo HC-05 (Fig. 2) implementa el perfil de puerto serie (SPP) de Bluetooth 2.0 y actúa como un puente transparente entre una conexión UART y un enlace inalámbrico [4]. Admite dos modos: el modo de datos, en el que retransmite la información recibida por UART (por defecto a 9600 baudios), y el modo de comandos AT, que permite configurar su nombre, rol maestro/esclavo y velocidad de comunicación. Dado que su entrada RX trabaja con niveles de 3,3 V, se requiere un divisor resistivo al conectarla a la salida TX del microcontrolador.



Figura 2: Módulo Bluetooth HC-05.

IV-E. *Teclado matricial*

Un teclado matricial 4×4, como el de la Fig. 3, dispone sus 16 pulsadores en la intersección de 4 filas y 4 columnas, reduciendo a 8 la cantidad de pines necesarios [5]. La lectura se realiza por barrido: se activa una fila a la vez (nivel bajo) mientras las columnas, configuradas como entradas con pull-up, se leen para detectar cuál de ellas fue llevada a nivel bajo por una tecla presionada. Como en todo pulsador mecánico, se requiere un tratamiento antirrebote.

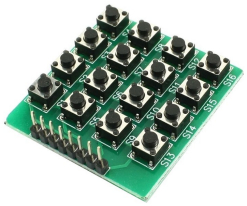


Figura 3: Teclado matricial 4×4.

IV-F. Servomotor SG90

El SG90 (Fig. 4) es un servomotor de posición que integra un motor DC, una caja reductora y un lazo de control interno con realimentación por potenciómetro [6]. Su eje se posiciona en un rango de aproximadamente 180° según el ancho de pulso de una señal de control de 50 Hz: pulsos de 1 ms, 1,5 ms y 2 ms corresponden, de forma aproximada, a los extremos y al centro de su recorrido.



Figura 4: Servomotor SG90.

IV-G. LEDs direccionables WS2812B

El WS2812B integra en un mismo encapsulado un LED RGB y su controlador, y permite encadenar múltiples unidades sobre una única línea de datos [7]. Cada LED recibe 24 bits en orden GRB (8 bits por color), retiene los primeros 24 bits que le llegan y retransmite el resto al siguiente de la cadena; un nivel bajo sostenido por más de 50 μs produce la actualización simultánea de todos los LEDs. Los bits se codifican por la duración del pulso alto dentro de un período de aproximadamente 1,25 μs (800 kHz), con tolerancias de $\pm$150 ns. A 16 MHz, esto equivale a unos 20 ciclos de reloj por bit, por lo que la rutina de transmisión debe tener una temporización exacta a nivel de instrucción y ejecutarse con las interrupciones deshabilitadas. En la matriz 8×8 utilizada (Fig. 5), los 64 LEDs se encuentran conectados en una única cadena, lo que obliga a convertir las coordenadas (fila, columna) en el índice correspondiente de dicha cadena.



Figura 5: Matriz de LEDs RGB 8×8 WS2812B.

IV-H. Sensor inercial MPU6050

El MPU6050, cuyo módulo se muestra en la Fig. 6, combina un acelerómetro y un giroscopio de tres ejes con conversores de 16 bits, y se comunica por I2C en la dirección 0x68 [8]. En reposo, el acelerómetro mide la proyección de la gravedad sobre cada eje, a partir de la cual se estiman los ángulos de inclinación:

$$\theta_{pitch} = \operatorname{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right) \tag{1}$$

$$\phi_{roll} = \operatorname{atan2}\left(a_y, a_z\right) \tag{2}$$

Esta estimación es precisa a largo plazo pero sensible a vibraciones, mientras que la integración de la velocidad angular del giroscopio es suave pero acumula deriva. El filtro complementario combina ambas fuentes ponderando cada una según su comportamiento en frecuencia:

$$\theta_k = \alpha\left(\theta_{k-1} + \omega_k\,\Delta t\right) + (1-\alpha)\,\theta_{acc,k} \tag{3}$$

donde α, típicamente entre 0,95 y 0,98, determina el peso relativo del giroscopio.



Figura 6: Sensor inercial MPU6050.

V. Materiales

- Microcontroladores ATmega328P / placas Arduino UNO.
- Protoboard(s) y cables de conexión.

- Fuente de alimentación de 5 V y fuente externa para motores.
- Lector RFID RC522 y tarjetas MIFARE Classic 1K.
- Teclado matricial 4×4.
- Pantallas LCD 16×2 con módulo I2C (PCF8574).
- Módulo Bluetooth HC-05 y teléfono celular con terminal Bluetooth.
- Matriz de LEDs RGB 8×8 WS2812B.
- Sensor inercial MPU6050.
- Sensor de temperatura y humedad DHT11.
- Sensor de presencia PIR HC-SR501.
- Fotorresistencias (LDR).
- Joystick analógico KY-023.
- Potenciómetros.
- Motor DC y driver L298N.
- Servomotor SG90.
- LEDs individuales.
- Resistencias.
- Pulsadores.
- Buzzers activos y pasivos.
- Microchip Studio, Proteus y PICSimLab.

VI. Procedimiento

VI-A. Problema A. Cajero automático

VI-A1. Selección de componentes y mapeo de pines:

- **Identificación:** lector RFID RC522 conectado al módulo SPI del microcontrolador (PB2 a PB5), con la línea de reset del lector en PB1. Las tarjetas MIFARE se identifican por su UID de 4 bytes [2], [3].
- **Entrada de datos:** teclado matricial 4×4, con las filas configuradas como salidas (PD4 a PD7) y las columnas como entradas con resistencia de pull-up interna (PD2, PD3, PB0 y PC3).
- **Visualización:** pantalla LCD 16×2 controlada mediante un expansor de E/S PCF8574 por bus I2C (PC4/PC5), en la dirección 0x27 [9].
- **Indicadores:** LED verde (PC0) y LED rojo (PC1), ambos con resistencia limitadora de 220 Ω, y un buzzer activo (PC2).
- **Depuración:** interfaz USART (PD0/PD1) a 9600 baudios conectada a una terminal serie.
- **Aprovechamiento del GPIO:** con el cristal externo de 16 MHz ocupando PB6/PB7 y las líneas de USART, SPI e I2C asignadas a sus pines dedicados, las cuatro columnas del teclado no entran en un único puerto, por lo que se repartieron entre los puertos D, B y C. La lectura se resolvió consultando individualmente cada pin.

Tabla I: Mapeo de pines del cajero automático

Señal	Pin AVR	Pin Arduino
RC522 SDA (SS)	PB2	D10
RC522 MOSI	PB3	D11
RC522 MISO	PB4	D12
RC522 SCK	PB5	D13
RC522 RST	PB1	D9
Teclado filas F1–F4	PD4–PD7	D4–D7
Teclado columna C1	PD2	D2
Teclado columna C2	PD3	D3
Teclado columna C3	PB0	D8
Teclado columna C4	PC3	A3
LCD SDA (PCF8574)	PC4	A4
LCD SCL (PCF8574)	PC5	A5
LED verde	PC0	A0
LED rojo	PC1	A1
Buzzer activo	PC2	A2
USART RX / TX	PD0 / PD1	D0 / D1

VI-A2. Lectura del UID y simulación del lector RFID:

- **Librerías de la cátedra:** la lectura de las tarjetas se realizó con las librerías provistas por los docentes [10]. La función `mfrc522_standard()` envía el comando REQA para detectar una tarjeta y, ante una respuesta, ejecuta el procedimiento de anticolisión que devuelve el UID. Las librerías se incluyeron como archivos fuente (`UART.c`, `SPI.c` y `RC522.c`), en ese orden, ya que `RC522.c` utiliza funciones de las otras dos.
- **Detección de tarjeta:** dado que la función de la librería no indica si la lectura fue exitosa y no modifica el búfer si la anticolisión no responde, el búfer se inicializa en cero antes de cada lectura y el UID se considera válido solo si es distinto de `00000000` y de `FFFFFFFF`. El lector se consulta cada 200 ms, y una tarjeta se considera retirada recién tras dos lecturas fallidas consecutivas, ya que, al no enviarse el comando de detención (HALT) tras cada lectura, se observó que la tarjeta responde únicamente a uno de cada dos comandos REQA.
- **Ausencia de modelo en Proteus:** al no existir un modelo del RC522 para Proteus, se reemplazó por el componente SPI Debugger configurado en modo esclavo. El UID de cada tarjeta de prueba se carga manualmente en la cola de transmisión del instrumento como una secuencia predefinida, y el microcontrolador lo recibe a través del bus SPI. Mediante la directiva de compilación `SIMULACION` se selecciona entre esta lectura y la lectura real con la librería, sin modificar el resto del programa.
- **Velocidad del bus en simulación:** la librería configura el SPI a $f_{osc}/2 = 8$ MHz, frecuencia a la que el SPI Debugger en modo esclavo perdía la alineación de los bits. Por ello, en modo simulación se reconfiguran los registros `SPCR` y `SPSR` luego de `spi_init()` para operar a $f_{osc}/128 = 125$ kHz,

sin modificar la librería. En el montaje físico el bus conserva la frecuencia original, admitida por el RC522 [3].

- **Ventana de lectura:** según el instante en que se encola la secuencia, el instrumento puede entregar los datos con un byte de retraso respecto del inicio de la transacción. Para independizar la lectura de este efecto, el microcontrolador lee una ventana de 8 bytes y toma como inicio del UID el primer byte distinto de `0xFF`, valor que el instrumento entrega cuando su cola está vacía. Si alguno de los cuatro bytes resultantes es `0xFF`, la lectura se descarta por estar incompleta.

VI-A3. Teclado matricial y visualización:

- **Barrido del teclado:** se coloca en nivel bajo una fila por vez y se leen las cuatro columnas; la tecla presionada se obtiene de una tabla indexada por fila y columna [5].
- **Antirrebote:** ante un cambio en el estado del teclado se espera 20 ms y se repite la lectura, aceptando la tecla solo si ambas coinciden. La función devuelve cada tecla una única vez por pulsación, evitando que la máquina de estados la procese repetidamente.
- **Teclado en simulación:** se utilizó el componente KEYPAD-SMALLCALC de Proteus, cuya disposición difiere de la del teclado físico. Conectando sus filas en el orden C, B, A, D a los pines PD4 a PD7, los dígitos coinciden con los del teclado físico y las teclas especiales quedan según la Tabla II, de modo que un único firmware sirve para ambos casos.
- **Configuración del puerto C:** la función `twi_init()` de la librería del LCD sobrescribe por completo los registros `DDRC` y `PORTC`, lo que deshabilitaba el pull-up de la columna C4 y encendía los LEDs. Por ello se invoca en primer lugar, y las inicializaciones del teclado y de los indicadores restablecen luego la configuración de esos pines.
- **Tiempo de ejecución del LCD:** la secuencia de inicialización de la librería finaliza con un comando de borrado, que el controlador HD44780 tarda 1,52 ms en ejecutar [11], sin esperar su finalización. Como consecuencia, se perdía el primer carácter escrito, por lo que se agregó una espera de 2 ms tras la inicialización. Para el resto de las actualizaciones, cada línea se reescribe completándola con espacios, evitando el uso del comando de borrado [12].

Tabla II: Equivalencia de teclas entre el teclado de Proteus y el teclado físico

Proteus	Físico	Función
−	A	Consultar saldo
×	B	Retiro
÷	C	Depósito
+	D	Salir / cancelar
ON/C	*	Borrar dígito
=	#	Confirmar

VI-A4. Almacenamiento persistente en EEPROM:

- **Estructura de cada cuenta:** cada cuenta ocupa 13 bytes: UID (4 bytes), PIN almacenado como cuatro caracteres ASCII para compararlo directamente con las teclas ingresadas (4 bytes), saldo como entero sin signo de 32 bits (4 bytes) e indicador de bloqueo (1 byte). La organización de la memoria se presenta en la Tabla III.
- **Inicialización:** en la dirección 0x00 se almacena un byte de verificación (`0xA5`). Si al encender el sistema este byte no está presente, se cargan las tres cuentas con sus valores por defecto, grabando el byte de verificación en último lugar para que una interrupción de la alimentación durante la carga provoque su repetición en el siguiente arranque.
- **Escritura:** se utilizó la función `eeprom_update_block()`, que solo graba los bytes cuyo valor cambió, prolongando la vida útil de la memoria, limitada a aproximadamente 100 000 ciclos de escritura por celda [1].
- **Reset de fábrica:** manteniendo presionada la tecla `D` al encender el sistema se restablecen las cuentas por defecto. En simulación, el mismo efecto se obtiene borrando la EEPROM simulada mediante la opción *Reset Persistent Model Data* de Proteus.
- **Montos:** con la autorización de los docentes, las operaciones se restringieron a montos múltiplos de 10, de hasta cinco dígitos. Para mostrar el saldo en el LCD se implementó una función propia de conversión de entero a texto, ya que en la configuración del entorno utilizada la función `ultoa()` de avr-libc no resultaba visible aun incluyendo `<stdlib.h>`, produciendo errores de declaración implícita y de enlace.

Tabla III: Organización de la memoria EEPROM

Dirección	Contenido
0x00	Byte de verificación (`0xA5`)
0x10	Cuenta 1 (13 bytes)
0x1D	Cuenta 2 (13 bytes)
0x2A	Cuenta 3 (13 bytes)

VI-A5. Modelado como máquina de estados:

- **Tipo de máquina:** se modeló como una máquina de Mealy, ya que los indicadores luminosos y sonoros

y la escritura en la EEPROM dependen tanto del estado actual como del evento recibido, y se asocian a las transiciones. La información mostrada en el LCD, en cambio, depende únicamente del estado actual, por lo que constituye una salida de tipo Moore; dado que las acciones determinantes del sistema ocurren en las transiciones, se adoptó la representación de Mealy.

- **Estados:** ESPERA (detección de tarjeta), PIN (autenticación), MENÚ (selección de operación), SALDO (consulta) y MONTO (ingreso del importe de un retiro o depósito).
- **Diagrama:** el diagrama completo, con las transiciones y las acciones asociadas, se presenta en la Fig. 7.
- **Implementación:** los estados se definieron mediante un tipo enumerado evaluado en una estructura `switch`, dentro de un lazo principal de 10 ms. Todos los tiempos del sistema se miden como múltiplos de ese período, por lo que el teclado nunca queda bloqueado por demoras prolongadas. La función `ir_a()` centraliza la entrada a cada estado: actualiza el estado, descarta la entrada parcial, reinicia el temporizador de inactividad y redibuja el LCD.
- **Mensajes transitorios:** los mensajes asociados a las transiciones se implementaron mediante un estado auxiliar que mantiene el mensaje y los indicadores durante 2 s y luego conduce al estado de destino.
- **Seguridad:** al tercer PIN incorrecto la cuenta se marca como bloqueada en la EEPROM, por lo que la tarjeta es rechazada incluso tras reiniciar el sistema. Además, si durante una sesión transcurren 30 s sin actividad en el teclado, la sesión se cierra automáticamente. El contador de intentos se reinicia en cada sesión para agilizar las pruebas; en un sistema real se llevaría por cuenta.
- **Lectura de tarjeta:** el lector solo se consulta en el estado ESPERA, de modo que la sesión no depende de que la tarjeta permanezca apoyada y, al finalizarla, no se inicia una nueva sesión hasta que la tarjeta se retira.

VI-A6. Metodología de desarrollo: El programa se desarrolló de forma incremental, con un commit de GitHub por cada etapa validada: lectura del UID por SPI con la librería de la cátedra, lectura del teclado con antirrebote, integración del LCD y los indicadores, almacenamiento de las cuentas en EEPROM y, por último, la máquina de estados completa. Cada etapa se verificó en Proteus mediante un bloque SIMULINO, utilizando el Virtual Terminal para la depuración por USART y el SPI Debugger como sustituto del lector RFID.

Inicialmente se evaluó emular el RC522 mediante un segundo microcontrolador configurado como esclavo SPI. Esta alternativa se descartó al adoptar la librería de la cátedra, ya que su frecuencia de bus de 8 MHz supera el máximo de $f_{osc}/4$ que admite un ATmega328P en modo esclavo [1].

Para el montaje físico se respetó el mismo mapeo de pines de la simulación. El RC522 se alimentó desde la salida de 3,3 V de la placa, dejando sin conectar su línea de interrupción, y el LCD se conectó mediante su módulo I2C con cuatro líneas (alimentación, masa, SDA y SCL), ajustando el contraste con el potenciómetro del módulo. Los UIDs de las tarjetas MIFARE disponibles se obtuvieron previamente mediante un programa auxiliar, incluido en el repositorio, que los transmite por USART. Este programa utiliza las funciones de acceso a registros de la librería de la cátedra para implementar la secuencia de REQA y anticolisión con verificación del byte BCC, sin los mensajes de depuración que emite `mfrc522_standard()` en cada llamada, lo que permitió identificar cada UID con claridad en el monitor serie. Los UIDs obtenidos se incorporaron a la tabla de cuentas por defecto, que se selecciona mediante la misma directiva `SIMULACION`, de modo que el firmware conserva los UIDs de prueba para la simulación y los reales para el hardware. Tras grabar el firmware, se ejecutó el reset de fábrica para cargar las nuevas cuentas en la EEPROM del microcontrolador físico.

VI-B. Problema B. Banda transportadora con arquitectura maestro–esclavo SPI

VI-B1. Arquitectura y selección de componentes:

- **Distribución de tareas:** el sistema se dividió entre dos ATmega328P (Arduino UNO a 16 MHz). El maestro concentra la lectura de sensores, la interfaz con el usuario y la toma de decisiones, mientras que el esclavo se limita a ejecutar sobre los actuadores las órdenes que recibe, sin lógica de proceso propia.
- **Entradas del maestro:** pulsador Start/Stop con resistencia de pull-up interna, potenciómetro de 10 kΩ como divisor de tensión para seleccionar la velocidad, LDR en divisor con una resistencia fija de 10 kΩ para detectar el paso de piezas y sensor DHT11 para la temperatura simulada del motor [13].
- **Visualización:** pantalla LCD 16×2 controlada mediante un expansor PCF8574 por bus I2C, en la dirección 0x27, con la librería provista por la cátedra [9].
- **Actuadores del esclavo:** motor DC de la banda accionado por un puente H L298N, con la habilitación por PWM y el sentido de giro fijo [14]; servomotor SG90 como pistón [6]; LED verde de marcha y LED rojo de falla, ambos con resistencia limitadora de 220 Ω, y un buzzer activo.

- **Conflicto con la librería del LCD:** la función `twi_init()` configura PC0 y PC1 como salidas en nivel alto. Por ese motivo, el potenciómetro y el LDR se asignaron a los canales ADC2 y ADC3, ya que en ADC0 y ADC1 sus señales quedarían forzadas a 5 V.

El mapeo de pines de ambos nodos se resume en la Tabla IV.

Tabla IV: Mapeo de pines del Problema B.

Señal	Pin AVR	Pin Arduino
Bus SPI (pin a pin entre ambos nodos)		
SS	PB2	D10
MOSI	PB3	D11
MISO	PB4	D12
SCK	PB5	D13
Maestro		
Pulsador Start/Stop	PD3	D3
DHT11	PD2	D2
Potenciómetro	PC2 (ADC2)	A2
LDR	PC3 (ADC3)	A3
LCD SDA / SCL	PC4 / PC5	A4 / A5
Esclavo		
L298N ENA (PWM)	PD6 (OC0A)	D6
L298N IN1 / IN2	PD7 / PB0	D7 / D8
Servo SG90	PB1 (OC1A)	D9
LED verde / rojo	PD4 / PD5	D4 / D5
Buzzer	PD3	D3

VI-B2. Modelado como máquina de estados:

- **Tipo de máquina:** la lógica del proceso se modeló en el maestro como una máquina de Mealy, en la que las salidas (los comandos SPI) están asociadas a las transiciones; por ejemplo, el paso de PARADO a MARCHA emite el comando START. El esclavo no tiene una máquina de estados de proceso: funciona como intérprete de comandos y solo conserva el estado de sus actuadores.
- **PARADO:** estado inicial, con el motor detenido. Permite ajustar la velocidad, que queda almacenada en el esclavo.
- **MARCHA:** la banda está en movimiento y la detección de piezas acciona el pistón y el contador.
- **EMERGENCIA:** estado enclavado, con el motor detenido y el pistón retraído. Distingue dos motivos, que el LCD muestra como EMR (sobretemperatura, 30 °C o más) y FAL (falla del sensor).
- **Rearme:** desde EMERGENCIA solo se vuelve a MARCHA pulsando Start, siempre que haya una lectura válida inferior a 28 °C. La separación entre ambos umbrales evita que el sistema oscile entre estados con temperaturas cercanas al límite.
- **Implementación:** los estados se representan con un tipo enumerado y el evento del pulsador se resuelve con una estructura `switch`. Las transiciones disparadas por el operador quedan condicionadas a la recepción del ACK. El diagrama completo se presenta en la Fig. 10.

VI-B3. Protocolo de comunicación SPI:

- **Configuración:** se utilizó el módulo SPI por hardware en modo 0, con una frecuencia de reloj de $f_{osc}/16 = 1$ MHz [1], [10]. El maestro controla la línea SS por software y el esclavo atiende la recepción mediante la interrupción `SPI_STC`.
- **Estructura de la trama:** cada comando se envía en una transacción de tres transferencias con SS en nivel bajo: `[COMANDO][VALOR][respuesta]`. Como el SPI es full-duplex y el esclavo solo puede transmitir durante una transferencia iniciada por el maestro, el tercer byte (de relleno, 0x00) es el que devuelve la confirmación. La rutina de interrupción del esclavo carga la respuesta en `SPDR` apenas recibe el VALOR, y el maestro espera 20 μs entre bytes para darle tiempo a hacerlo.
- **Comandos:** se definieron cinco comandos, detallados en la Tabla V.
- **Validación y confirmación:** el esclavo verifica que el código de comando exista y que el valor esté dentro del rango admitido. Si la trama es válida responde ACK (0x06) y la ejecuta; en caso contrario responde NACK (0x15) y la descarta.
- **Reintentos:** si no recibe ACK, el maestro reintenta el envío hasta tres veces, y solo cambia de estado cuando la orden fue confirmada, de modo que nunca asume que una orden se ejecutó. La línea MISO del maestro tiene la resistencia de pull-up interna activa, así que con el esclavo desconectado se lee 0xFF y se interpreta como NACK.
- **Excepción del paro de emergencia:** ante una emergencia, el maestro cambia de estado aunque no reciba confirmación y reenvía el comando en cada lectura de temperatura hasta obtener el ACK, priorizando la seguridad.
- **Resincronización:** el esclavo lleva un índice de la posición del byte dentro de la trama. El programa principal lo reinicia cada vez que encuentra SS en nivel alto, con las interrupciones deshabilitadas durante la comprobación, de manera que una trama incompleta no desfase las siguientes.

Tabla V: Comandos del protocolo SPI.

Comando	Código	Valor	Acción en el esclavo
START	0x01	0	Pone en marcha el motor a la velocidad almacenada, enciende el LED verde y apaga la alarma.
STOP	0x02	0	Detiene el motor, retrae el pistón y apaga los indicadores.
PISTON	0x03	0	Extiende el servo durante 500 ms (solo en marcha).
VEL	0x04	0–100	Almacena la velocidad (%) y actualiza el PWM si la banda está en marcha.
EMERG	0x05	1 / 2	Detiene el motor y retrae el pistón. Motivo 1 (sobretemperatura): LED rojo y buzzer intermitentes; motivo 2 (falla del sensor): LED rojo fijo.

VI-B4. Adquisición de sensores:

- **Base de tiempo:** el Timer0 en modo CTC genera una interrupción cada 1 ms, que incrementa un contador. A partir de él, el programa principal ejecuta cada tarea con su propio período y sin retardos bloqueantes: pulsador cada 10 ms, LDR cada 20 ms, potenciómetro cada 50 ms y DHT11 cada 2 s.
- **Pulsador:** el antirrebote se resolvió por software, aceptando un cambio de nivel recién tras tres muestras consecutivas iguales (30 ms) y actuando solo en el flanco de presión.
- **Potenciómetro:** el ADC trabaja con referencia AVcc y prescaler 128 (125 kHz). Cada lectura es el promedio de cuatro conversiones, escalado a un porcentaje de 0 a 100. Para no saturar el bus con tramas causadas por el ruido del conversor, el comando VEL solo se envía cuando la lectura varía al menos dos puntos o alcanza exactamente 0 o 100.
- **LDR:** la resistencia fija de 10 kΩ forma con el LDR un divisor de tensión, ya que el ADC mide tensión y no resistencia:

$$V_{PC3} = 5\,\mathrm{V} \cdot \frac{R_{LDR}}{R_{LDR} + 10\,\mathrm{k\Omega}} \tag{4}$$

  Cuando una pieza tapa la luz, la resistencia del LDR aumenta y la lectura sube. La detección usa dos umbrales con histéresis (presencia con 750 cuentas o más y liberación con 675 o menos), y cada cambio requiere tres lecturas consecutivas (60 ms), de modo que una pieza se cuenta una única vez aunque la señal oscile cerca de un umbral. Los umbrales se calibraron en simulación: con 10,1 lx se midieron 3,18 V (651 cuentas) y con 5,1 lx, 3,79 V (776 cuentas). La detección se ejecuta siempre, pero el conteo y el accionamiento del pistón solo ocurren con la banda en marcha.
- **DHT11:** se reutilizó la rutina de lectura desarrollada en el laboratorio anterior, con verificación de checksum. El período de 2 s respeta el intervalo mínimo de 1 s entre lecturas del sensor. Tres lecturas fallidas consecutivas se consideran una falla del sensor. La lectura deshabilita las interrupciones durante unos 4 ms, lo que no afecta al resto del programa, porque el maestro genera el reloj SPI y las demás tareas toleran ese retraso.

VI-B5. Control de actuadores en el esclavo:

- **Ejecución de órdenes:** la rutina de interrupción solo almacena la trama y activa una bandera; el programa principal es quien ejecuta el comando, para mantener la interrupción breve.
- **Motor de la banda:** el Timer0 genera un PWM rápido no invertido en OC0A con prescaler 64, lo que da una frecuencia de unos 976 Hz. Para velocidades mayores que cero, el ciclo útil se calcula como `OCR0A` $=$ `PWM_MIN` $+\, v\,(255 -$ `PWM_MIN`$)/100$, donde `PWM_MIN` compensa el rozamiento estático del motorreductor, que no arranca con ciclos útiles bajos. Con velocidad nula o con la banda detenida, la salida OC0A se desconecta y el pin queda en nivel bajo, porque en modo PWM rápido un valor de `OCR0A` igual a cero todavía produce un pulso de un ciclo. El esclavo almacena la velocidad aunque la banda esté detenida, de modo que al recibir START arranca directamente a la velocidad seleccionada.
- **Pistón:** el Timer1 en modo 14 (PWM rápido con tope en `ICR1`), con prescaler 8, da una resolución de 0,5 μs por cuenta [1]. Con `ICR1` $= 39999$ se obtiene el período de 20 ms (50 Hz) que requiere el servo, y `OCR1A` alterna entre 2000 cuentas (1 ms, reposo) y 4000 cuentas (2 ms, extendido).
- **Temporización:** como los Timer0 y Timer1 están ocupados por los PWM, el Timer2 en modo CTC aporta la base de 1 ms del esclavo. Con ella, el pistón vuelve al reposo a los 500 ms sin bloquear la recepción de comandos. Ante un STOP o un paro de emergencia, se retrae de inmediato.
- **Señalización:** en marcha normal se enciende el LED verde; con la banda detenida, ambos LEDs quedan apagados; ante una falla del sensor, el LED rojo queda fijo, y ante una sobretemperatura, el LED rojo y el buzzer conmutan juntos cada 250 ms.

VI-B6. Robustez de la comunicación con el LCD:

- **Problema detectado:** tanto en simulación como en el montaje físico, luego de varios refrescos de la pantalla aparecían caracteres erróneos en la primera línea, y en algunos casos el maestro quedaba bloqueado, sin responder al pulsador.
- **Frecuencia del bus I2C:** la librería configuraba el bus a 400 kHz, mientras que el PCF8574 está especificado hasta 100 kHz [9]. Se redujo la frecuencia a 100 kHz.

- **Esperas con límite de tiempo:** las funciones de `twi.c` esperaban indefinidamente el estado esperado del bus, por lo que una sola falta de confirmación del PCF8574 bloqueaba todo el programa, incluido el paro de emergencia. Se modificaron para abandonar la transferencia tras unos 2 ms y reiniciar el módulo TWI, de modo que una falla del bus afecte solo a la pantalla.
- **Inicialización del LCD:** la secuencia de la librería envía bytes completos durante el arranque, lo que no permite recuperar el controlador si quedó desfasado en medio byte dentro del modo de 4 bits. Se reemplazó por la secuencia del fabricante, que envía tres veces el nibble 0x3 y luego 0x2 [11], precedida por una espera de 50 ms para el encendido del display. Esta rutina se ejecuta al arrancar y cada 2 s, después de cada lectura del DHT11, para corregir cualquier desfase en poco tiempo. El costo es una ocupación del maestro de unos 70 ms cada 2 s, inferior al tiempo de paso de una pieza frente al LDR.
- **Formato de la primera línea:** durante la depuración se comprobó que una cadena fija se mostraba siempre correctamente, mientras que la generada con `snprintf` a partir de los valores variables producía caracteres erróneos. Por eso, la primera línea se arma a partir de la plantilla fija `T:-- V:--- P:000`, en la que una función propia escribe los dígitos de la temperatura, la velocidad y el contador en posiciones fijas.

VI-B7. Metodología de desarrollo:

- **Diseño previo de la máquina de estados:** antes de escribir el código se definieron los estados del proceso (PARADO, MARCHA y EMERGENCIA), los eventos que provocan cada transición y el criterio de enclavamiento y rearme de la emergencia. El tipo enumerado con los tres estados se incorporó desde la primera etapa, y cada etapa posterior agregó las transiciones y salidas que le correspondían.
- **Desarrollo incremental:** a partir de ese modelo, el programa se construyó en cinco etapas, cada una validada antes de pasar a la siguiente y registrada como un commit en el repositorio:
  1. Enlace SPI maestro–esclavo con trama `[CMD][VALOR]`, ACK y botón Start/Stop.
  2. Control de velocidad de la banda por potenciómetro (ADC) y PWM en el esclavo.
  3. Detección de piezas con LDR (ADC con histéresis), contador y accionamiento del pistón (servo).
  4. Monitoreo de temperatura con DHT11, paro de emergencia enclavado y señalización con buzzer.
  5. Robustez del LCD I2C: inicialización según la hoja de datos, resincronización periódica y esperas con límite de tiempo en el bus.
- **Simulación:** se utilizó Proteus, con dos ATmega328P a 16 MHz conectados por SPI. El LCD se armó con los modelos PCF8574 y LM016L, ya que el componente JHD-2X16-I2C no es compatible con la librería. El motor se simuló con el modelo L298 y un motor DC; el modelo del L298 requiere conectar a GND las entradas SENSA y SENSB para que circule corriente por la carga. El pistón se simuló con el modelo MOTOR-PWMSERVO, y tanto el LDR como el DHT11 se ajustaron de forma interactiva (nivel de iluminación y temperatura).
- **Implementación física:** el L298N se alimentó con una fuente externa de 12 V, retirando el puente de ENA para controlar la habilitación por PWM. El servomotor se alimentó desde la salida de 5 V del regulador del L298N, para no cargar la alimentación de las placas con sus picos de corriente de arranque. Se unieron los GND de ambas placas, de la fuente y del driver, ya que el SPI requiere una referencia común. Los umbrales del LDR calibrados en simulación y el valor inicial de `PWM_MIN` (80) resultaron adecuados con la iluminación del laboratorio y el motorreductor utilizado, por lo que no fue necesario modificarlos. El DHT11 se calentó con una fuente de calor externa, a distancia, hasta superar el umbral de emergencia.

VI-C. Problema C. Sistema de Alarma Antirrobo con Supervisión Inalámbrica

VI-C1. Selección de componentes y mapeo de pines:

- **Unidad de procesamiento:** microcontrolador ATmega328P montado sobre placa Arduino UNO, operando a una frecuencia de reloj de 16 MHz.
- **Comunicación inalámbrica:** módulo Bluetooth HC-05 conectado a los pines de la interfaz USART por hardware (PD0/RX y PD1/TX), configurado a 9600 baudios. Dado que la entrada RX del módulo opera con niveles lógicos de 3,3 V, se implementó un divisor resistivo para adaptar de forma segura la salida TX del microcontrolador. Las líneas USART asociadas al HC-05 debían desconectarse temporalmente durante la carga del firmware para evitar interferencias con el puerto USB de la placa.
- **Sensores de intrusión y perimétricos:** sensor de presencia infrarroja pasiva PIR (HC-SR501) para la detección de movimiento y una fotorresistencia (LDR) dispuesta en un divisor de tensión con resistencia fija para el monitoreo lumínico o de aperturas.
- **Visualización:** pantalla LCD 16×2 controlada mediante un expansor de E/S PCF8574 a través del bus I2C (PC4/SDA y PC5/SCL), en la dirección 0x27.
- **Actuadores e indicadores:** buzzer activo para la emisión de alertas sonoras e indicadores luminosos

(LEDs verde y rojo) equipados con resistencias limitadoras de 220 Ω.

Tabla VI: Mapeo de pines del sistema de alarma antirrobo

Señal	Pin AVR	Pin Arduino
HC-05 RX / TX (USART)	PD0 / PD1	D0 / D1
Sensor PIR	PD3	D3
LDR (ADC2)	PC2	A2
LED de armado	PD4	D4
LED de alarma	PD5	D5
Buzzer activo	PD6	D6
LCD SDA / SCL (PCF8574)	PC4 / PC5	A4 / A5

El mapeo de pines se resume en la Tabla VI.

VI-C2. Comunicación inalámbrica y recepción de comandos:

- **Puente transparente:** el módulo HC-05 opera en modo de datos, retransmitiendo de manera transparente la información recibida desde un dispositivo móvil o terminal Bluetooth hacia el microcontrolador mediante el puerto serie UART.
- **Interpretación de comandos:** se estableció un conjunto de caracteres de control (por ejemplo, 'A' para armado del sistema, 'D' para desarmado, 'M' para modo manual, 'S' para silenciamiento, 'U' para modo automático y 'F' para forzar disparo) procesados mediante rutinas de ejecución secuencial diseñadas para evitar bloqueos en el bucle principal.
- **Notificación al celular:** el sistema informa de manera bidireccional los eventos y la zona afectada mediante tramas estructuradas en formato de texto clave-valor delimitadas por punto y coma (por ejemplo, `ALARMA:ON;ZONA:VENTANA` o `ALARMA:DISPARADA;ZONA:PIR`). Estas tramas se transmiten a través del puerto serie al producirse los cambios de estado en la máquina de estados.

VI-C3. Adquisición de señales de sensores:

- **Sensor PIR:** conectado a una entrada digital (PD3) para capturar transiciones a nivel alto ante la presencia de movimiento en el área vigilada.
- **Fotorresistencia (LDR):** conectada al canal analógico ADC2 (PC2) mediante un divisor de tensión, su señal es procesada mediante un algoritmo de suavizado basado en el promedio aritmético de $N = 10$ muestras consecutivas, que atenúa las fluctuaciones rápidas de la señal (sombras momentáneas o ruido de conversión) y evita disparos falsos.

VI-C4. Modelado como máquina de estados:

- **Tipo de máquina:** el comportamiento del sistema se modeló como una máquina de estados finitos de tipo Mealy, donde las acciones de control sobre los actuadores (buzzer e indicadores) y la transmisión de respuestas dependen tanto del estado actual como de los eventos de entrada (comandos inalámbricos o señales de los sensores).
- **Estados operativos:**
  - `ST_DESARMADO`: estado inicial de reposo, en el cual el sistema inicializa sus periféricos y permanece a la espera de órdenes de armado.
  - `ST_ARMADO_AUTO`: supervisión activa de los sensores de intrusión (PIR y LDR).
  - `ST_ALARMA`: activación de las señales de sirena y parpadeo de los indicadores ante la detección de un evento o la ejecución del comando 'F'.
  - `ST_SILENCIADO`: permite apagar el aviso acústico manteniendo el estado de supervisión o control activo.
  - `ST_MANUAL`: modo de operación en el cual el operador ejerce el control remoto directo desde la aplicación del celular para armar, desarmar, silenciar o forzar la alarma de manera interactiva.

VI-C5. Metodología de desarrollo: El software se desarrolló de forma incremental mediante seis commits en GitHub:

1. Configuración de macros, pines base y controladores USART y ADC.
2. Declaración de variables de estado globales e inicialización de la pantalla LCD.
3. Configuración de registros de E/S y adición de la lectura de los sensores LDR y de movimiento.
4. Implementación de la lógica de detección de flancos para pulsadores locales.
5. Adición de la ISR de recepción UART y de la función `procesar_comando()`.
6. Completado de la lógica de gestión de alarmas, actuadores y renderizado en LCD.

La validación se realizó en los entornos virtuales Proteus y PICSimLab. Para el montaje físico se respetó el esquema de asignación, conectando el módulo HC-05 en PD0/PD1, el sensor PIR en PD3, la fotorresistencia LDR en PC2, los LEDs indicadores en PD4/PD5, el buzzer en PD6 y la pantalla LCD a través del bus I2C (PC4/SDA y PC5/SCL).

VI-D. Problema D. Juego de Memoria de Secuencias

VI-D1. Selección de componentes y mapeo de pines:

- **Interfaz de salida:** matriz RGB 8×8 WS2812B comandada por una sola línea de datos serie en PD6 (D6).
- **Entrada analógica:** módulo joystick analógico (KY-023), con el eje X conectado al canal ADC0 (PC0/A0) y el eje Y al canal ADC1 (PC1/A1).
- **Entrada digital:** pulsador de selección/confirmación integrado en el joystick (SW), conectado a PD2 (D2) con la resistencia de pull-up interna del ATmega328P.

- **Indicador sonoro:** buzzer pasivo de 5 V conectado a PB1 (D9/OC1A), controlado mediante PWM generado por hardware.
- **Depuración:** transmisión serie USART en PD1 (TX) a 9600 baudios (8N1), para monitorear la secuencia, las coordenadas y los estados del juego.

Tabla VII: Mapeo de pines del Juego Simón Dice RGB

Señal / Módulo	Pin AVR	Pin Arduino	Función
WS2812B DIN	PD6	D6	Datos serie para 64 LEDs RGB
Joystick VRx	PC0 (ADC0)	A0	Lectura analógica eje X
Joystick VRy	PC1 (ADC1)	A1	Lectura analógica eje Y
Joystick SW	PD2	D2	Pulsador con *pull-up* interno
Buzzer	PB1 (OC1A)	D9	Salida de tonos mediante Timer1 PWM
USART TX	PD1	D1	Salida de depuración serie (9600 bps)

VI-D2. Temporización, generación de tonos y protocolo WS2812B:

- **Base de tiempo (Timer0):** configurado en modo CTC con prescaler de 64 y `OCR0A = 249`, lo que genera una interrupción (`TIMER0_COMPA_vect`) cada 1 ms a 16 MHz. En cada interrupción se incrementa un contador global de milisegundos, que utiliza la función `millis()`.
- **Síntesis de audio (Timer1):** configurado en modo Fast PWM de 16 bits (modo 14, con TOP en ICR1) y prescaler de 8. Para cada nota de frecuencia f se recalcula ICR1 según la Ec. (5) y se fija el ciclo de trabajo en 50 % (`OCR1A = ICR1/2`). Se definieron una frecuencia para cada color del juego (330, 392, 440 y 523 Hz) y un tono grave de 110 Hz para indicar el error.

$$\mathrm{ICR1} = \frac{F_{\mathrm{CPU}}}{8f} - 1 \tag{5}$$

- **Protocolo WS2812B:** cada bit se codifica con un pulso en alto de $0{,}4$ μs para el '0' y de $0{,}8$ μs para el '1', con una tolerancia de ± 150 ns [7]. Por esa razón, la rutina de envío `ws_byte()` se escribió con instrucciones en lenguaje ensamblador (`asm volatile`). Durante el envío de los 64 LEDs (192 bytes en formato GRB) se deshabilitan las interrupciones (`cli()`) para no alterar la temporización, y al final se agrega una pausa de 80 μs para aplicar el RESET. El cálculo del índice en la cadena admite matrices lineales y matrices con disposición en serpentina (`SERPENTINA`). Además, la imagen se reenvía cada 100 ms (`REFRESCO_MS`) aunque no cambie, porque en la simulación se observó que, sin ese refresco, el cursor dejaba de verse con el joystick en reposo.

VI-D3. Lectura de periféricos y acondicionamiento de señales:

- **Acondicionamiento del joystick:** el potenciómetro del joystick no queda perfectamente centrado en reposo, por lo que se implementó una zona muerta de ± 150 cuentas alrededor del punto medio (512 cuentas del ADC). La velocidad del cursor se limitó con la constante `PASO_CURSOR_MS` (200 ms), de modo que mantener el joystick inclinado no desplace el cursor de forma descontrolada.
- **Detección de eventos del pulsador:** la función `btn_evento()` lee el pin PD2 cada 10 ms para filtrar el rebote mecánico y distingue tres eventos:
  1. `BTN_PRESIONADO`: se genera en el instante en que se aprieta el pulsador. Durante el turno del jugador confirma la posición del cursor.
  2. `BTN_CORTO`: se genera al soltar el pulsador antes de 1000 ms. En el modo libre cambia el color del cursor de forma aleatoria.
  3. `BTN_LARGO`: se genera cuando el pulsador se mantiene apretado durante `T_PULSO_LARGO` = 1000 ms. Inicia una nueva partida.
- **Generador pseudoaleatorio:** se utiliza un generador de tipo xorshift de 16 bits (variable `rng`). Su semilla se combina mediante XOR, al arrancar, con las lecturas de los canales ADC0 y ADC1 del joystick y, al iniciar cada partida, con el valor de `millis()` en el momento de la pulsación larga. Cada elemento nuevo de la secuencia incluye las coordenadas $(f, c) \in [0..7] \times [0..7]$ y un color $col \in [0..3]$, y se verifica que su posición sea distinta de la del elemento anterior.

VI-D4. Almacenamiento persistente en EEPROM: La EEPROM interna del ATmega328P se organiza de la siguiente forma:

- **0x00:** byte de validación (`EE_MAGIC` = 0xA5). Si no está presente al encender, el récord y el último nivel se inicializan en 0.
- **0x01:** récord histórico de niveles completados (`EE_RECORD`).
- **0x02:** nivel alcanzado en la última partida (`EE_ULTIMO`).

Antes de cada escritura se compara el valor almacenado con el nuevo (`ee_leer(dir) == dato`) y solo se escribe si cambió, con lo que se evitan escrituras innecesarias.

VI-D5. Modelado e implementación de la máquina de estados: El comportamiento del sistema se modeló con una máquina de estados finitos de tipo Mealy, porque las acciones de salida (tonos del buzzer, patrones de la matriz y escrituras en la EEPROM) dependen no solo del estado actual, sino también de los eventos de entrada: el joystick, el pulsador y los temporizadores. Los estados se definieron con un tipo enumerado (`typedef enum`):

- **`ST_RECORD`:** estado inicial al energizar. Muestra el récord almacenado en la EEPROM con LEDs azules durante 2000 ms (`T_RECORD`).

- **ST_LIBRE:** modo libre. El cursor se desplaza por la matriz y cambia de color con `BTN_CORTO`. Una pulsación larga (`BTN_LARGO`) inicia una partida.
- **ST_MUESTRA:** el sistema ilumina en orden cada posición de la lista `seq[]` durante 500 ms (`T_ENCENDIDO`), con pausas de 200 ms (`T_PAUSA`) y el tono correspondiente a cada color.
- **ST_JUGADOR:** el jugador mueve el cursor y confirma con el pulsador (`BTN_PRESIONADO`). El sistema compara la posición con la del elemento esperado de la secuencia.
- **ST_ACIERTO:** animación de nivel superado, en cuatro fases no bloqueantes, con destellos verdes y tonos ascendentes. Al terminar se agrega un elemento a la secuencia.
- **ST_ERROR:** animación de fallo, con destellos rojos y un tono grave de 110 Hz. Al terminar se actualiza el récord en la EEPROM, si se superó, y el sistema vuelve al modo libre.

La máquina se ejecuta dentro del lazo principal (`for(;;)`) mediante una estructura `switch(estado)`. Para no bloquear la CPU durante las animaciones, los tiempos se resolvieron comparando el valor de `millis()` con la marca de tiempo del estado (`t_estado`), de modo que el joystick y el pulsador se siguen leyendo en todo momento.

VI-D6. Metodología de desarrollo: El programa se desarrolló de forma incremental, en seis etapas. Cada una se validó en simulación y se registró como un commit en el repositorio de GitHub:

1. Parámetros, paleta de colores y base de tiempo con Timer0.
2. Driver de la matriz WS2812B y manejo del framebuffer.
3. Lectura del joystick por ADC, cursor y botón con antirrebote.
4. Buzzer por PWM con Timer1 y UART de depuración.
5. Generación de la secuencia y máquina de estados del juego.
6. Récord y última partida en EEPROM.

Cada etapa se verificó en PICSimLab con una placa Arduino UNO, dos controles deslizantes como joystick, un pulsador, un buzzer y el terminal virtual (VTerm) para la depuración por USART. Para el montaje físico se mantuvo el mismo mapeo de pines de la simulación (Tabla VII).

VI-E. Problema E. Burbuja de Inclinación con MPU6050

VI-E1. Selección de componentes y mapeo de pines:

- **Unidad de procesamiento:** microcontrolador ATmega328P integrado en una placa Arduino UNO o compatible, operando a una frecuencia de reloj de 16 MHz.
- **Sensor de inclinación:** acelerómetro de 3 ejes MPU6050 conectado a través del bus I2C por hardware (pines PC4/SDA y PC5/SCL), operando con rutinas de inicialización y recuperación por timeout.
- **Matriz de visualización espacial:** matriz de LEDs RGB direccionables WS2812B (8×8) conectada al pin digital PB3 del microcontrolador. Este pin se seleccionó por disponibilidad y conveniencia de distribución física en la protoboard. La transmisión de datos se realiza mediante manipulación de puertos en ensamblador en línea (`__asm__ volatile`).
- **Visualización alfanumérica:** pantalla LCD 16×2 controlada mediante un expansor de E/S PCF8574 a través del bus I2C. La dirección se selecciona mediante una directiva de compilación condicional (`#define MODO_SIMULACION`), que conmuta entre 0x27 para el hardware real y 0x23 para la simulación en PICSimLab.
- **Actuadores e indicadores de control:** buzzer activo conectado al pin digital PB2 para la emisión de alertas acústicas, y dos pulsadores con resistencias de pull-up internas habilitadas en los pines PB0 (modo HOLD) y PB1 (calibración).

Tabla VIII: Mapeo de pines del nivel de burbuja

Señal	Pin AVR	Pin Arduino
Pulsador HOLD	PB0	D8
Pulsador CALIB	PB1	D9
Buzzer activo	PB2	D10
WS2812B DIN	PB3 (MOSI)	D11
I2C SDA (MPU6050 y PCF8574)	PC4	A4
I2C SCL (MPU6050 y PCF8574)	PC5	A5

El mapeo de pines se resume en la Tabla VIII.

VI-E2. Gestión de interfaz de usuario y calibración:

- **Modo de retención (HOLD):** permite congelar temporalmente la visualización en la pantalla LCD y mantener la posición estática de la burbuja sin requerir la lectura constante del sensor inercial.
- **Calibración no volátil:** el botón de calibración captura las lecturas brutas actuales de los ejes X e Y del MPU6050 para establecer los nuevos offsets de nivel cero, los cuales son almacenados de forma permanente en la memoria EEPROM interna del ATmega328P mediante rutinas seguras (`eeprom_update_word()`).

VI-E3. Adquisición y procesamiento de señales inerciales:

- **Lectura en bloque:** se realiza una solicitud secuencial por I2C hacia los registros de salida del acelerómetro (`ACCEL_XOUT_H`) para capturar de manera sincrónica los datos de aceleración lineal.

- **Modelado trigonométrico:** a partir de los valores leídos y compensados con los offsets, se calculan los ángulos de inclinación longitudinal (pitch) y transversal (roll) utilizando funciones trigonométricas estándar de la biblioteca matemática de C (atan2).
- **Filtro digital (media móvil y suavizado temporal):** para mitigar el ruido inherente del sensor y las vibraciones mecánicas del entorno, se implementó un algoritmo de suavizado mediante un filtro de media móvil con una ventana de N=10 muestras, estabilizando las lecturas angulares antes de ser procesadas por el sistema.
- **Mapeo espacial de inclinación ($\pm 30^\circ$ a celdas 0–7):** el rango dinámico de inclinación operativa se estableció en $\pm 30^\circ$ respecto a la horizontal. Mediante un algoritmo de escalado lineal con división entera, cada incremento de ángulo se traduce a celdas discretas de 0 a 7 sobre la matriz de 8×8, aplicando funciones de saturación por software (*clamp*) para asegurar que, si la inclinación supera los $\pm 30^\circ$, la coordenada se bloquee exactamente en los bordes extremos (0 o 7) sin desbordar la matriz.
- **Lógica cromática de la burbuja:** el color de la burbuja cambia dinámicamente según la distribución de sus ejes:
  - **Azul** (B = 255): indica nivelación perfecta en el centro de la matriz en la coordenada $(4, 4)$ ($\pm 3^\circ$).
  - **Rojo** (R = 255): predomina cuando la inclinación longitudinal en el eje X (roll) es mayor.
  - **Verde** (G = 255): predomina cuando la inclinación transversal en el eje Y (pitch) es mayor.
  - **Amarillo** (R = 255, G = 255): se activa cuando coexisten inclinaciones mixtas equivalentes en ambos ejes.

VI-E4. Modelado como máquina de estados y control espacial:

- **Tipo de máquina:** el comportamiento del sistema se modeló formalmente como una máquina de estados finitos de tipo Mealy.
- **Estados operativos:**
  - `ST_NORMAL`: estado de reposo y operación continua, donde se adquieren datos del MPU6050, se calculan los ángulos, se actualiza la LCD y se refresca la matriz WS2812B.
  - `ST_HOLD`: estado de congelamiento de la posición actual en pantalla mediante la pulsación del botón de retención.
  - `ST_LIMITE`: estado de alerta activado automáticamente cuando la burbuja alcanza los bordes de la matriz (0 o 7). Ante este evento, el sistema emite un pitido intermitente en el buzzer (con una frecuencia de activación sincronizada al ciclo de refresco de 80 ms) y genera un parpadeo de la burbuja en la matriz para advertir al usuario sobre el límite del rango de nivelación.
  - `ST_CALIBRACION`: estado destinado a la captura de la referencia de nivel y su almacenamiento no volátil en la EEPROM.

VI-E5. Metodología de desarrollo: El software se desarrolló de forma incremental mediante seis commits en GitHub:

1. Configuración inicial y bus TWI seguro.
2. Implementación del driver LCD I2C.
3. Driver de la matriz WS2812B en ensamblador en línea.
4. Inicialización y lectura del acelerómetro MPU6050.
5. Configuración de E/S, EEPROM y secuencia de bienvenida.
6. Lógica del bucle principal (filtrado digital, trigonometría, estados HOLD, LÍMITE y CALIBRACIÓN, y actuadores).

La validación se realizó en el entorno virtual PICSimLab, utilizando la directiva de compilación `MODO_SIMULACION` para alternar la dirección I2C del expansor PCF8574 (0x23 en simulación y 0x27 en hardware real). Para el montaje físico se respetó el esquema de pines: alimentación a 5 V, matriz WS2812B en PB3 (MOSI), pulsadores de control en PB0 y PB1, buzzer en PB2 y el bus I2C compartido (PC4/SDA y PC5/SCL) para el MPU6050 y la LCD. La calibración de los offsets quedó respaldada de forma permanente en la memoria EEPROM del ATmega328P.

VII. Resultados

VII-A. Problema A. Cajero automático

VII-A1. Diagrama de máquina de estados: La Fig. 7 presenta el diagrama completo de la máquina de estados implementada, con sus cinco estados (ESPERA, PIN, MENÚ, SALDO y MONTO), las transiciones entre ellos y las acciones asociadas a cada una (indicadores luminosos y sonoros y escritura en la EEPROM). Los estados que conforman una sesión se agrupan en un recuadro, desde el cual el cierre por inactividad conduce al estado ESPERA. Este diagrama fue la base directa sobre la que se implementó la lógica de transición del programa, y permite verificar que cada camino descrito en la metodología se corresponde con una transición real del sistema.

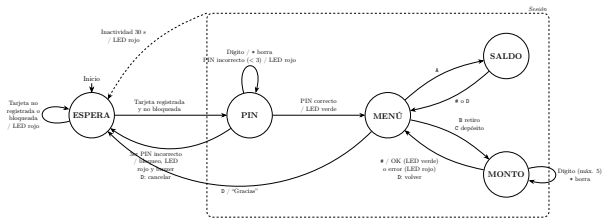


Figura 7: Diagrama de la máquina de estados del cajero automático (Problema A).

VII-A2. Simulación: La Fig. 8 muestra el circuito simulado en Proteus, compuesto por el bloque SIMULINO, el teclado matricial, el LCD con su expansor PCF8574, los indicadores, el Virtual Terminal y el SPI Debugger utilizado como sustituto del lector RFID. Sobre este circuito se ejecutó el ciclo completo de operación del cajero, cargando en el SPI Debugger las secuencias correspondientes a las tres cuentas registradas y a una tarjeta no registrada.

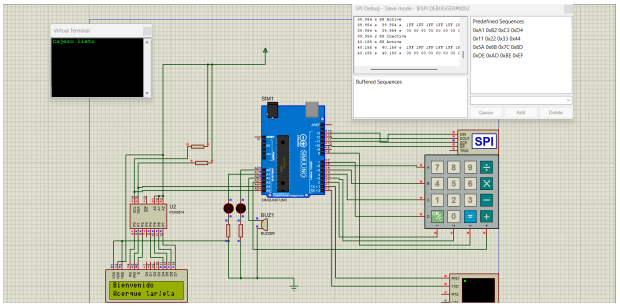


Figura 8: Simulación del cajero automático en Proteus.

VII-A3. Validación en simulación e implementación física:

- **Identificación de tarjetas:** el microcontrolador recibió correctamente por SPI los UIDs cargados en el SPI Debugger, independientemente del instante en que se encolaba cada secuencia, gracias a la lectura mediante ventana de 8 bytes. Las tarjetas registradas condujeron a la solicitud de PIN, mientras que la tarjeta no registrada fue rechazada con el mensaje correspondiente y el encendido del LED rojo.
- **Teclado y visualización:** las 16 teclas del teclado fueron reconocidas una única vez por pulsación y en correspondencia con la Tabla II. El LCD mostró correctamente los mensajes de cada estado, incluido el primer carácter tras la corrección del tiempo de inicialización.
- **Autenticación y bloqueo:** un PIN correcto dio acceso al menú con el encendido del LED verde, y cada PIN incorrecto informó el número de intento. Al tercer intento fallido la cuenta quedó bloqueada, y la tarjeta fue rechazada directamente en ESPERA en los intentos posteriores.
- **Operaciones:** la consulta de saldo, los retiros y los depósitos actualizaron el saldo de la cuenta según lo esperado. Los montos no múltiplos de 10 y los retiros superiores al saldo disponible fueron rechazados con el mensaje de error correspondiente y el encendido del LED rojo.
- **Inactividad:** transcurridos 30 s sin actividad en el teclado durante una sesión, el sistema la cerró automáticamente y regresó al estado ESPERA.
- **Persistencia:** tras detener y reiniciar la simulación, los saldos modificados y el estado de bloqueo de las cuentas se conservaron, confirmando el almacenamiento en la EEPROM simulada.
- **Indicadores:** los LEDs verde y rojo respondieron en cada transición según lo previsto. El buzzer no emitió sonido en Proteus, por lo que en simulación su salida se verificó reemplazándolo temporalmente por un LED; en el montaje físico se confirmó su activación al bloquearse una cuenta.

La Tabla IX resume las pruebas realizadas y el entorno en que se validó cada una.

Tabla IX: Resultados de validación del cajero automático

Prueba	Resultado	Validado en
Tarjeta registrada	Solicitud de PIN	Simulación y físico
Tarjeta no registrada	Mensaje de rechazo y LED rojo	Simulación y físico
PIN correcto	Acceso al menú y LED verde	Simulación y físico
PIN incorrecto	Indicación del intento y LED rojo	Simulación y físico
Tercer PIN incorrecto	Bloqueo en EEPROM, LED rojo y buzzer	Simulación y físico
Tarjeta bloqueada	Rechazo directo en ESPERA	Simulación y físico
Consulta de saldo	Visualización del saldo actual	Simulación y físico
Retiro válido	Descuento y almacenamiento del saldo	Simulación y físico
Depósito válido	Incremento y almacenamiento del saldo	Simulación y físico
Monto no múltiplo de 10	Mensaje de monto inválido	Simulación y físico
Retiro mayor al saldo	Mensaje de saldo insuficiente	Simulación y físico
Inactividad de 30 s	Cierre de sesión	Simulación y físico
Reinicio del sistema	Saldos y bloqueos conservados	Simulación y físico

VII-A4. Circuito físico: La Fig. 9 muestra el circuito armado físicamente, con el lector RC522, el teclado matricial, el LCD con módulo I2C, los indicadores luminosos y el buzzer conectados al microcontrolador según el mapeo de la Tabla I. Sobre este circuito se repitieron las pruebas con tres tarjetas MIFARE registradas y una cuarta no registrada, y se confirmó el mismo comportamiento verificado en simulación, incluyendo la persistencia de los saldos y del bloqueo de cuentas

tras desconectar la alimentación. La única diferencia de operación entre ambos entornos fue la disposición de las teclas especiales, ya que el teclado físico corresponde directamente a las funciones asignadas, sin la equivalencia de la Tabla II.

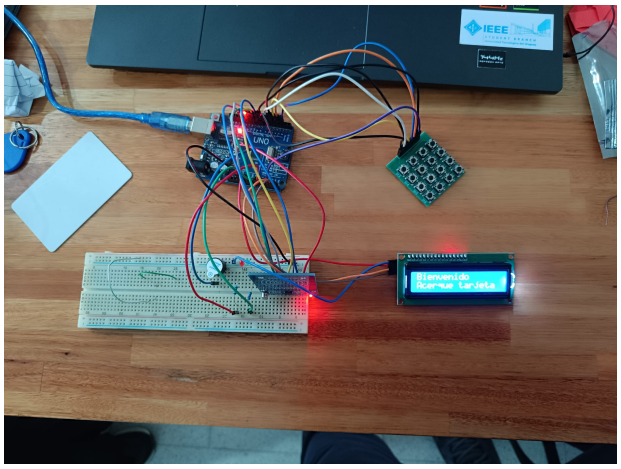


Figura 9: Circuito físico del cajero automático.

VII-B. Problema B. Banda transportadora con arquitectura maestro–esclavo SPI

VII-B1. Diagrama de máquina de estados: La Fig. 10 presenta la máquina de estados del maestro. Al encenderse, el sistema envía el comando STOP para llevar al esclavo a un estado conocido y queda en PARADO. Desde allí, el pulsador alterna entre PARADO y MARCHA, y cada transición emite el comando correspondiente. Los lazos representan eventos que generan comandos sin cambiar de estado: la variación del potenciómetro (VEL) en cualquier estado y la detección de piezas (PISTON) únicamente en MARCHA. La sobretemperatura o la falla del sensor llevan al sistema a EMERGENCIA desde cualquiera de los dos estados operativos, y en ese estado el pulsador solo tiene efecto cuando se cumple la condición de rearme.

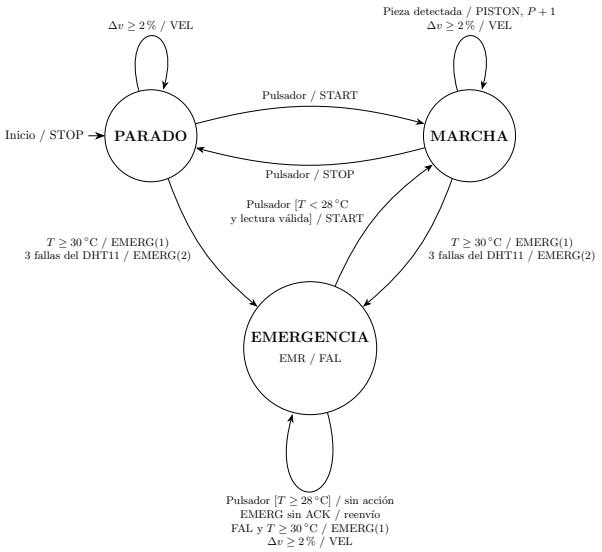


Figura 10: Diagrama de la máquina de estados del maestro de la banda transportadora (Problema B). Las transiciones disparadas por el pulsador se concretan solo si el esclavo responde ACK.

VII-B2. Simulación: La Fig. 11 muestra el circuito simulado en Proteus, con el maestro a la derecha (pulsador, potenciómetro, LDR, DHT11 y LCD) y el esclavo a la izquierda (L298N con el motor, servo, LEDs y buzzer), vinculados por las cuatro líneas del bus SPI.

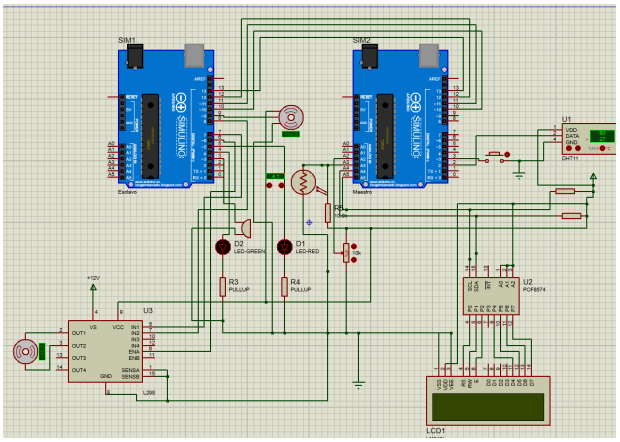


Figura 11: Simulación del Problema B en Proteus.

VII-B3. Validación:

- **Enlace SPI:** al encender, el LCD mostró la confirmación del STOP inicial. Al desconectar la línea MISO y pulsar Start, el maestro informó NACK tras los tres reintentos y permaneció en PARADO, lo que comprobó que no cambia de estado sin confirmación.
- **Control de velocidad:** con la banda detenida, el LCD actualizó la velocidad al girar el potencióme-

tro, sin que el motor se moviera. Al pulsar Start, el motor arrancó a la velocidad seleccionada y siguió las variaciones del potenciómetro. Con 0 % se detuvo sin salir de MARCHA, y el comando STOP lo detuvo con cualquier velocidad.

- **Detección de piezas:** al reducir la iluminación del LDR de 10,1 a 5,1 lx con la banda en marcha, el contador aumentó en uno y el servo se extendió durante 500 ms. Manteniendo el LDR oscurecido no se registraron conteos adicionales, y en PARADO las piezas fueron ignoradas. Un STOP con el pistón extendido lo retrajo de inmediato.
- **Paro de emergencia:** al llevar el DHT11 a 30 °C, tanto en MARCHA como en PARADO, el sistema pasó a EMERGENCIA en la lectura siguiente: el motor se detuvo y el LED rojo y el buzzer comenzaron a conmutar. Con 29 °C el pulsador no tuvo efecto, y con 27 °C rearmó el proceso, que retomó la marcha a la velocidad del potenciómetro.
- **Falla del sensor:** al desconectar la línea de datos del DHT11, el LCD indicó FAL a los 6 s aproximadamente y el LED rojo quedó encendido de forma fija, sin buzzer. Al reconectarlo con una temperatura inferior a 28 °C, el pulsador volvió a rearmar el sistema.
- **Robustez del LCD:** en el montaje físico, al desconectar la línea SDA con la banda en marcha, el pulsador y el motor siguieron respondiendo, y al reconectarla la pantalla recuperó el texto correcto en menos de 2 s.

Todas las pruebas descritas se repitieron sobre el montaje físico, con resultados equivalentes a los obtenidos en simulación. La Tabla X resume la verificación de cada funcionalidad requerida.

Tabla X: Resumen de validación del Problema B.

Funcionalidad	Validado en
Arranque y paro del proceso (START / STOP)	Simulación y físico
Detección de pieza y accionamiento del pistón	Simulación y físico
Control de velocidad por potenciómetro (PWM)	Simulación y físico
Monitoreo de temperatura y paro de emergencia	Simulación y físico
Señalización de marcha, falla y emergencia	Simulación y físico
Visualización de sensores, contador y comandos	Simulación y físico
Comunicación SPI con confirmación (ACK)	Simulación y físico
Tolerancia a fallas del bus I2C	Físico

VII-B4. Circuito físico: La Fig. 12 muestra el montaje físico de la banda transportadora, con los dos Arduino UNO vinculados por el bus SPI, el driver L298N alimentado por la fuente externa y el resto de los sensores y actuadores sobre la protoboard. Sobre este circuito se confirmó el comportamiento verificado en simulación, incluyendo el paro de emergencia por calentamiento real del DHT11.

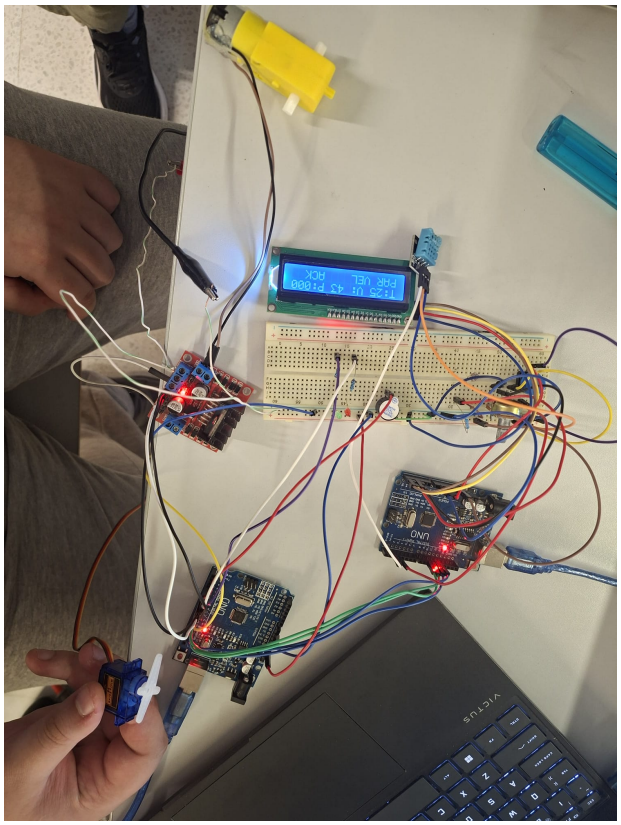


Figura 12: Circuito físico de la banda transportadora.

VII-C. Problema C. Sistema de Alarma Antirrobo con Supervisión Inalámbrica

VII-C1. Diagrama de máquina de estados: La lógica de control quedó formalizada mediante el diagrama de la máquina de estados de tipo Mealy, el cual detalla las transiciones entre los estados de reposo, armado automático, alarma, silenciamiento y modo manual, asociando cada cambio de estado a las acciones correspondientes sobre los indicadores y actuadores, e incluyendo el comando 'F' para forzar el disparo de manera remota.

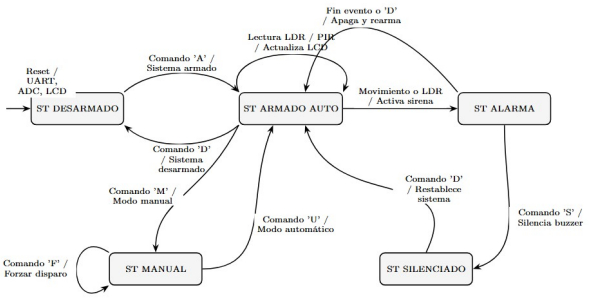


Figura 13: Diagrama de la máquina de estados del sistema de alarma antirrobo.

VII-C2. Simulación: En el entorno virtual se observó, mediante el uso de la terminal virtual, la recepción de tramas de comandos a través de la interfaz Bluetooth simulada, así como un comportamiento coherente del sistema ante la manipulación de los estados del sensor PIR y la fotorresistencia LDR. La pantalla LCD 16×2 reflejó el estado operativo actual y los mensajes informativos.

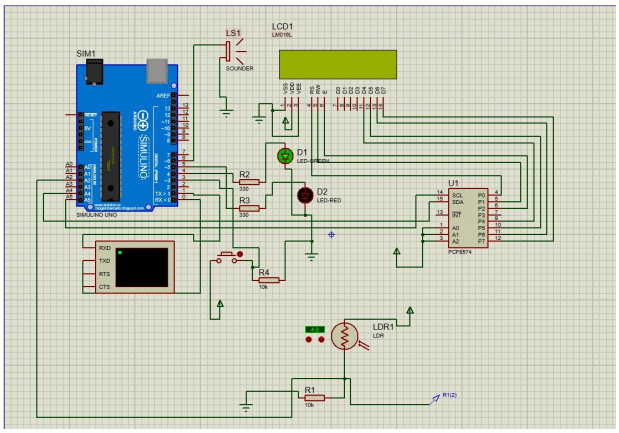


Figura 14: Simulación del sistema de alarma antirrobo en Proteus.

VII-C3. Circuito físico y montaje experimental: Los componentes del sistema se ensamblaron sobre placas protoboard empleando un microcontrolador ATmega328P integrado en una placa Arduino UNO, alimentada a 5 V.

- **Interfaz de comunicación inalámbrica:** el módulo Bluetooth HC-05 se conectó a los pines de transmisión y recepción serie, implementando el divisor resistivo necesario en su línea de entrada para adecuar los niveles lógicos de tensión sin comprometer la integridad de la transmisión de datos.
- **Periféricos de visualización y señalización:** la pantalla LCD 16×2 se acopló mediante su módulo expansor I2C (PCF8574), requiriendo únicamente cuatro líneas de conexión (alimentación, masa, SDA y SCL), lo que simplificó significativamente el cableado físico. Asimismo, los indicadores luminosos con sus respectivas resistencias de limitación y el buzzer activo se ubicaron de forma accesible para verificar de manera visual y acústica las transiciones de estado.

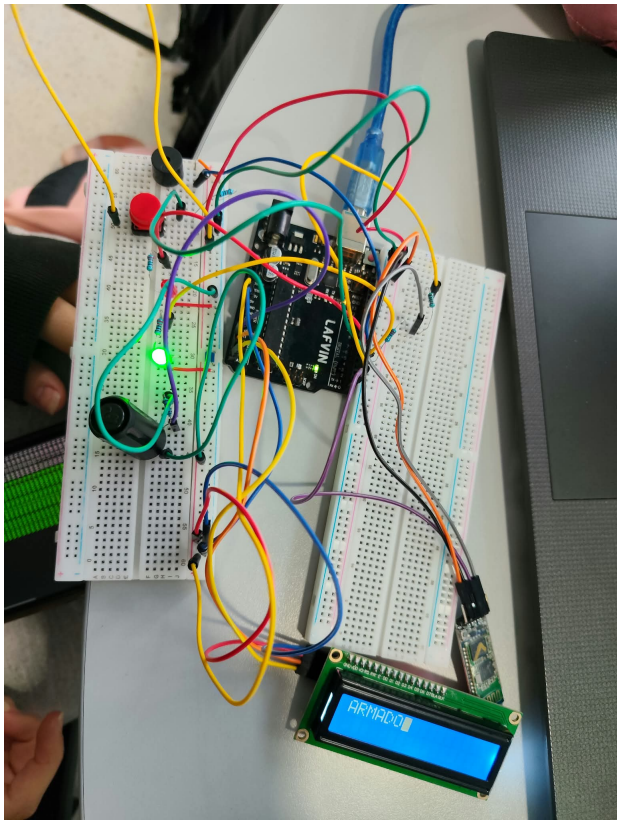


Figura 15: Montaje físico del sistema de alarma antirrobo.

VII-C4. Validación en hardware real: Durante los ensayos experimentales realizados sobre el prototipo físico, el sistema mostró un desempeño coherente con el verificado en el entorno de simulación:

- **Estabilidad del enlace inalámbrico:** las tramas de comandos enviadas de forma remota (como el armado, desarmado y silenciamiento) fueron interpretadas por el microcontrolador a través del puerto USART, observándose una adecuada recepción de los datos durante la operación.
- **Respuesta de los sensores de intrusión:** el sensor PIR y la red perimetral basada en fotorresistencia respondieron a los estímulos físicos aplicados, activando el estado de alarma y ejecutando las rutinas de señalización programadas en la máquina de estados.
- **Robustez ante perturbaciones:** la implementación de rutinas de filtrado temporal y muestreo múltiple

Tabla XI: Resultados de validación del sistema de alarma antirrobo

Prueba	Comportamiento esperado	Validado en
Arranque	Configuración de puertos, USART e I2C y mensaje inicial en el LCD.	Simulación y físico
Armado (‘A’)	Transición a `ST_ARMADO_AUTO` y envío de trama de confirmación al celular.	Simulación y físico
Detección de intrusión	Transición a `ST_ALARMA`, encendido del buzzer y envío de la trama `ALARMA:DISPARADA; ZONA:PIR`.	Simulación y físico
Forzado (‘F’)	Disparo manual de la alarma desde el celular.	Simulación y físico
Silenciamiento (‘S’)	Apagado del buzzer manteniendo la supervisión activa.	Simulación y físico
Modo manual (‘M’)	Armado, desarmado, silenciamiento o forzado controlados desde el celular.	Simulación y físico
Desarmado (‘D’)	Retorno a `ST_DESARMADO` y apagado de los indicadores.	Simulación y físico

(N = 10 muestras) en las señales de entrada permitió atenuar las fluctuaciones rápidas de la señal (sombras momentáneas o ruido de conversión), favoreciendo la estabilidad en la transición entre los diferentes modos operativos del sistema.

La Tabla XI resume las pruebas realizadas y el entorno en que se validó cada una.

VII-D. Problema D. Juego de Memoria de Secuencias

VII-D1. Diagrama de máquina de estados: La Fig. 16 presenta el diagrama de la máquina de estados de tipo Mealy del juego Simón Dice RGB, con sus seis estados (`ST_RECORD`, `ST_LIBRE`, `ST_MUESTRA`, `ST_JUGADOR`, `ST_ACIERTO` y `ST_ERROR`). Cada transición se rotula como *evento / acción*: los eventos provienen de los temporizadores del Timer0, del joystick y de su pulsador, y las acciones son los cambios en la matriz, los tonos del buzzer y las escrituras en la EEPROM.

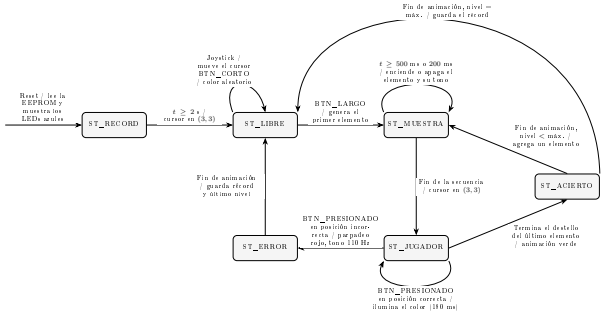


Figura 16: Diagrama de la máquina de estados del juego Simón Dice RGB.

Tabla XII: Resultados de la validación del juego Simón Dice RGB

Prueba	Comportamiento esperado	Validado en
Inicio y récord	Muestra en la matriz un LED azul por cada nivel del récord guardado en la EEPROM, durante 2 s.	Simulación y físico
Modo libre	El cursor se mueve sin salir del rango $[0..7]$ y cambia de color con una pulsación corta.	Simulación y físico
Inicio de partida	Una pulsación larga (> 1000 ms) genera la secuencia del nivel 1.	Simulación y físico
Reproducción de la secuencia	Encendido secuencial de los LEDs, 500 ms cada uno, con su tono asociado.	Simulación y físico
Turno del jugador	Lectura de las coordenadas (f, c) y confirmación de la casilla con respuesta visual y sonora.	Simulación y físico
Animación de acierto	Destellos verdes con tonos ascendentes y avance de nivel.	Simulación y físico
Animación de error	Destellos rojos, tono grave de fallo y regreso al modo libre.	Simulación y físico
Persistencia en EEPROM	Al cortar la alimentación se conserva el récord y se muestra al volver a energizar.	Simulación y físico
Depuración por USART	Envío por el puerto serie (9600 baudios) de las coordenadas de la secuencia generada.	Simulación y físico

VII-D2. Validación del sistema y resultados de pruebas: El sistema se verificó en simulación y en el montaje físico, con el ATmega328P, la matriz WS2812B, el módulo joystick KY-023 y el buzzer pasivo. La Tabla XII resume las pruebas funcionales y el entorno en que se validó cada una.

VII-D3. Análisis del desempeño:

- **Temporización del WS2812B:** la rutina de envío en ensamblador utiliza 20 ciclos de reloj por bit. A partir de ese conteo, el pulso en alto dura $0{,}375$ μs para el ‘0’ y $0{,}8125$ μs para el ‘1’, y el nivel bajo dura $0{,}875$ μs y $0{,}4375$ μs, respectivamente. Todos estos valores quedan a menos de 150 ns de los tiempos nominales del datasheet [7]. En las pruebas no se observaron destellos ni cambios de color no deseados en la matriz.
- **Acondicionamiento del joystick:** con la zona muerta de ± 150 cuentas alrededor del centro (512) no se observaron movimientos espontáneos del cursor con el joystick en reposo.
- **Generación de audio:** el Timer1 en modo Fast PWM genera la señal del buzzer por hardware, por lo que el lazo principal solo reprograma ICR1 y OCR1A cuando cambia la nota.
- **Uso de la EEPROM:** al escribir solo cuando el valor cambia, se evitan escrituras repetidas sobre la misma celda y se reduce el número de ciclos de escritura.

VII-D4. Simulación en PICSimLab: El circuito completo se simuló en PICSimLab con una placa Arduino UNO (ATmega328P) y las piezas del menú *Spare parts*.

El terminal virtual (VTerm), conectado a la línea de transmisión USART (PD1/TX) a 9600 baudios, permitió ver el nivel activo, el récord guardado y las coordenadas (f, c, col) de cada elemento de la secuencia (Fig. 17).

- **Verificación de periféricos:** el joystick se modeló con dos controles deslizantes conectados a PC0 (ADC0) y PC1 (ADC1), y un pulsador en PD2. La matriz WS2812B de 8×8 se verificó comprobando que la posición y el color de cada LED encendido coincidieran con las coordenadas (f, c, col) impresas en el VTerm. En el modelo de la matriz, la fila 0 corresponde a la fila inferior.
- **Generación sonora:** la salida PWM del Timer1 en PB1 (D9) se conectó a un buzzer simulado, y se comprobó el cambio de frecuencia para la nota de cada color (330, 392, 440 y 523 Hz).
- **Pruebas de persistencia:** se modificó el récord en partidas exitosas y se reinició la simulación. Al encender, el programa recuperó el valor guardado.

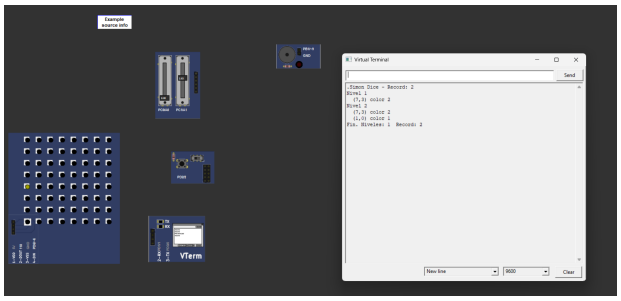


Figura 17: Simulación del juego Simón Dice en PICSimLab.

VII-D5. Montaje e implementación física: Los componentes se montaron en protoboard con una placa Arduino UNO (ATmega328P) alimentada a 5 V por USB, y se respetó el mapeo de pines de la Tabla VII (Fig. 18).

- **Alimentación de la matriz:** la matriz se alimentó desde los pines 5V y GND de la placa. Se limitó la intensidad de cada canal por software a 40 de 255 (`BRILLO = 40`), unos 16 % de la máxima. Como estimación teórica, tomando 20 mA por canal a plena intensidad, cada canal encendido consume unos 3 mA. La animación que enciende los 64 LEDs en un solo color (verde o rojo) requeriría entonces unos 200 mA, por debajo de los 500 mA que puede entregar un puerto USB. La corriente no se midió.
- **Buzzer:** se utilizó un buzzer pasivo conectado entre PB1 (D9/OC1A) y masa, excitado con la señal cuadrada que genera el Timer1.
- **Botón de reinicio (opcional):** se instaló un pulsador entre el pin RESET y masa, para reiniciar el programa, que al arrancar vuelve a mostrar el récord.

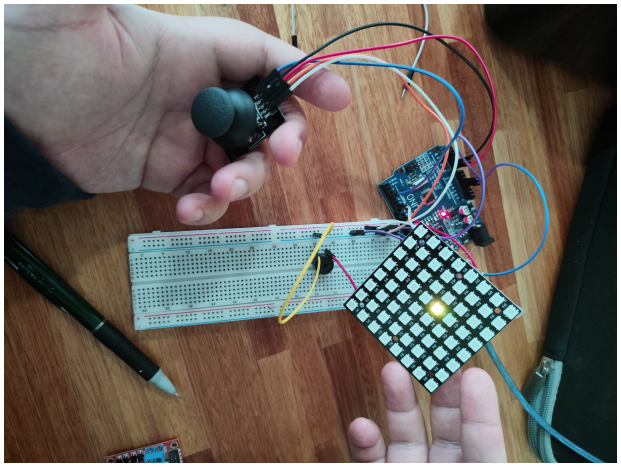


Figura 18: Montaje físico del juego Simón Dice.

VII-D6. Validación en hardware real: En los ensayos físicos el cursor respondió con fluidez al joystick, y la zona muerta de ± 150 cuentas evitó los movimientos causados por el centrado imperfecto del potenciómetro. El envío serie hacia la matriz de 64 LEDs no produjo parpadeos ni distorsiones de color visibles. Además, el récord se conservó en la EEPROM después de desconectar la alimentación.

VII-E. Problema E. Burbuja de Inclinación con MPU6050

VII-E1. Diagrama de máquina de estados: La lógica de control quedó formalizada mediante el diagrama de la máquina de estados finitos (Fig. 19), detallando las transiciones entre los estados `ST_NORMAL`, `ST_HOLD`, `ST_LIMITE` y `ST_CALIBRACION`.

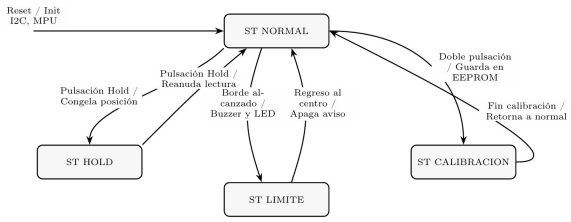


Figura 19: Diagrama de la máquina de estados de la burbuja de inclinación.

VII-E2. Simulación: En el entorno virtual (PICSimLab) se verificó con éxito la respuesta dinámica de la burbuja ante variaciones de orientación del MPU6050, la persistencia de datos en la EEPROM y la fluidez de la pantalla LCD I2C.

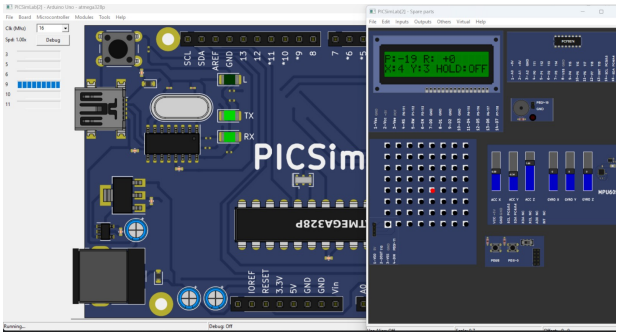


Figura 20: Simulación de la burbuja de inclinación en PICSimLab.

VII-E3. Circuito físico y montaje experimental: El sistema se ensambló sobre placas protoboard interconectadas a la placa con microcontrolador ATmega328P a 5 V, conectando el módulo expansor PCF8574 para la LCD, el sensor MPU6050, la matriz WS2812B, el buzzer y los pulsadores de control.

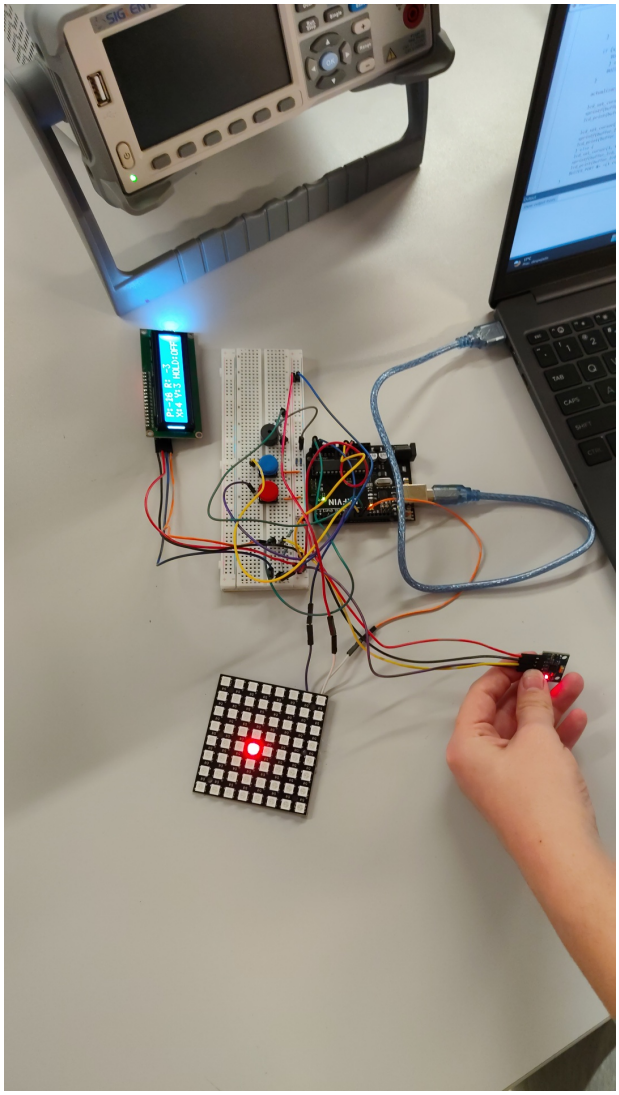


Figura 21: Montaje físico de la burbuja de inclinación.

VII-E4. Validación en hardware real: Durante las pruebas experimentales sobre el prototipo físico, el sistema demostró un desempeño coherente con el comportamiento verificado en la simulación.

- **Estabilidad del bus I2C:** con el uso de funciones protegidas por timeout y la dirección física adecuada, la pantalla LCD operó sin bloqueos.
- **Respuesta de los sensores y actuadores:** el acelerómetro respondió adecuadamente a las inclinaciones físicas aplicadas, reflejando el desplazamiento de la burbuja en la matriz y activando las alarmas acústicas al llegar a los límites perimetrales.

La Tabla XIII resume las pruebas realizadas y el entorno en que se validó cada una.

Tabla XIII: Resultados de validación del nivel de burbuja

Prueba	Comportamiento esperado	Validado en
Nivelado ($0° \pm 3°$)	Burbuja en la coordenada $(4, 4)$ en color azul; el LCD muestra `P:+0, R:+0`.	Simulación y físico
Inclinación mayor a 30° en X	Saturación en el borde ($X = 0$ o $X = 7$), buzzer activo y burbuja en rojo.	Simulación y físico
Modo HOLD	Congelamiento de los valores en el LCD y en la matriz, con indicador `HOLD:ON`.	Simulación y físico
Calibración	Guardado de los offsets en la EEPROM y mensaje "Calibrado!" en el LCD.	Simulación y físico

VIII. Conclusiones

El laboratorio permitió desarrollar cinco sistemas embebidos sobre el ATmega328P programados en lenguaje C, que integran comunicación con dispositivos externos, lectura de sensores, control de actuadores y almacenamiento persistente. Todos se validaron tanto en simulación como sobre el circuito físico, lo que confirma que el objetivo general se cumplió. El modelado de cada problema como una máquina de estados finitos, implementada en un lazo principal no bloqueante, fue el eje común de los cinco diseños: permitió atender en paralelo entradas, temporizaciones y comunicaciones, y mantener una correspondencia directa entre el diagrama y el código.

En cuanto a la comunicación, se trabajó con cuatro protocolos de naturaleza distinta, y cada uno exigió entender su funcionamiento más allá de la configuración de registros. En SPI, el carácter full-duplex y el hecho de que el esclavo solo pueda transmitir durante una transferencia iniciada por el maestro condicionaron el diseño de la confirmación de cada comando. En el lector RC522, el comportamiento observado de las tarjetas solo pudo explicarse a partir de la máquina de estados de la norma ISO/IEC 14443-A. En el WS2812B, la tolerancia de ± 150 ns obligó a resolver la transmisión en ensamblador en línea y con las interrupciones deshabilitadas, mostrando el límite de lo que puede garantizarse desde C. El bus I2C, compartido entre el LCD y el MPU6050, y el enlace UART sobre Bluetooth completaron el conjunto, y permitieron comprobar que un mismo microcontrolador puede atender varios periféricos y protocolos de forma simultánea.

La memoria EEPROM se empleó para conservar datos que deben sobrevivir a la desconexión de la alimentación: saldos y bloqueos de cuentas, el récord del juego y los offsets de calibración del sensor inercial. Su uso requirió definir de antemano la organización de las direcciones y escribir solo cuando el valor cambia, debido a la vida útil limitada de cada celda.

Por último, el acondicionamiento de las señales resultó tan determinante como la lógica de control. El promedio de muestras en las lecturas analógicas, la zona muerta del joystick y del nivel de burbuja, y el filtrado de las lecturas del acelerómetro permitieron obtener un comportamiento estable a partir de sensores ruidosos, evitando disparos falsos y oscilaciones en la visualización. Del mismo modo, los errores del LCD sobre el bus I2C pusieron en evidencia la importancia de la robustez de la comunicación: fue necesario respetar la frecuencia especificada por el fabricante y limitar el tiempo de espera de cada transferencia, para que una falla en un periférico secundario no bloqueara funciones críticas como el paro de emergencia. En conjunto, el laboratorio mostró que el desarrollo de un sistema embebido confiable depende tanto de dominar los periféricos y protocolos como de anticipar las limitaciones físicas de los componentes reales.

Apéndice A
Repositorio

El código fuente desarrollado en este laboratorio, junto con el historial de versiones de cada problema, se encuentra disponible en el siguiente repositorio público de GitHub:

https://github.com/gonzalogorosito-collab/Labs-MIcro-2026

Los videos que demuestran el funcionamiento de los sistemas desarrollados en este laboratorio se encuentran disponibles en la siguiente carpeta de Google Drive:

https://drive.google.com/drive/folders/12WoXbdIAJQaREWmjxrDE6npyOL0DOoRD?usp=drive_link

Referencias

[1] *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash – Datasheet*, Microchip Technology Inc., 2015, DS40002061A.

[2] *MF1S50yyX/V1 MIFARE Classic EV1 1K – Product data sheet*, NXP Semiconductors, 2018. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MF1S50YYX_V1.pdf

[3] *MFRC522 Standard performance MIFARE and NTAG frontend – Product data sheet*, NXP Semiconductors, 2016, rev. 3.9. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

[4] *HC-05 Bluetooth Module – Datasheet*, ITead Studio, 2010.

[5] J. Martínez and M. Díaz, "EEPROM – HC-SR04 – teclado matricial," Notas de clase, Tecnologías de Microprocesamiento, Ing. Mecatrónica, ITR Suroeste, UTEC, 2026.

[6] *SG90 9 g Micro Servo – Datasheet*, Tower Pro, s.f.

[7] *WS2812B Intelligent control LED integrated light source – Datasheet*, Worldsemi Co., Ltd., 2013. [Online]. Available: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf

[8] *MPU-6000 and MPU-6050 Product Specification*, InvenSense Inc., 2013, rev. 3.4. [Online]. Available: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf

[9] *PCF8574 Remote 8-Bit I/O Expander for I2C Bus – Datasheet*, Texas Instruments, 2015, SCPS068J.

[10] J. Martínez and M. Díaz, "SPI – RC522," Notas de clase, Tecnologías de Microprocesamiento, Ing. Mecatrónica, ITR Suroeste, UTEC, 2026.

[11] *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver – Datasheet*, Hitachi, Ltd., 1998, ADE-207-272(Z).

[12] J. Martínez and M. Díaz, “Comunicación I2C,” Notas de clase, Tecnologías de Microprocesamiento, Ing. Mecatrónica, ITR Suroeste, UTEC, 2026.
[13] *DHT11 Humidity & Temperature Sensor – Datasheet*, Aosong Electronics Co., Ltd., 2010.
[14] *L298 Dual Full-Bridge Driver – Datasheet*, STMicroelectronics, 2000.